

DECCAN EDUCATION SOCIETY'S — DES PUNE UNIVERSITY, PUNE

Data Structures (ETCS300) — End Semester Detailed Answer Key

December 2025 | Max Marks: 50 | Time: 2 Hours | Attempt ANY TWO from each question

Q.1 UNIT I — Algorithm Analysis & Asymptotic Notations (Attempt any 2)

[10 Marks]

a) Frequency / Step Count Method — Nested Loop Analysis

[5MM]

Given Code:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= i; j++) {  
        x = x + 1;  
    }  
}
```

Step 1 — Identify each statement and count how many times it executes:

Statement	When it runs	Frequency f(n)	Steps per run	Total fxs
int i=1 (init)	Once at start	1	1	1
i≤n (outer check)	After each i increment	n+1	1	n+1
i++ (outer incr)	Each outer loop pass	n	1	n
int j=1 (init)	Once per outer iteration	n	1	n
j≤i (inner check)	i+1 times per outer iter	$1+2+3+\dots+(n+1) = \frac{n(n+1)}{2} + n$	1	$\frac{n(n+1)}{2} + n$
j++ (inner incr)	i times per outer iter	$1+2+3+\dots+n = \frac{n(n+1)}{2}$	1	$\frac{n(n+1)}{2}$
x = x + 1	Inner body	$1+2+3+\dots+n = \frac{n(n+1)}{2}$	1	$\frac{n(n+1)}{2}$

Step 2 — Sum all total steps:

$$\begin{aligned}T(n) &= 1 + (n+1) + n + n + [\frac{n(n+1)}{2} + n] + \frac{n(n+1)}{2} + \frac{n(n+1)}{2} \\&= 1 + n+1 + n + n + \frac{n^2}{2} + \frac{n}{2} + n + \frac{n^2}{2} + \frac{n}{2} + \frac{n^2}{2} + \frac{n}{2} \\&= 3\frac{n(n+1)}{2} + 4n + 2\end{aligned}$$

Step 3 — Drop constants and lower-order terms:

Dominant term = $n^2 \rightarrow T(n) = O(n^2)$

Case	Complexity	Reason
Best Case	$O(n^2)$	No branching in this code — runs same regardless
Average Case	$O(n^2)$	Inner loop runs i times always
Worst Case	$O(n^2)$	Total iterations = $\frac{n(n+1)}{2} \approx n^2$

◆ Examiner's Note: Examiners look for: (1) a statement frequency table, (2) summation formula, (3) final $O()$ answer. Show all three.

b) Binary Search — Best / Average / Worst Case Time Complexity

[5MM]

Given Code:

```
int binarySearch(int arr[], int n, int key) {  
    int low = 0, high = n-1;  
    while (low <= high) {  
        int mid = (low + high) / 2;
```

```

    if (arr[mid] == key) return mid;
    else if (arr[mid] < key) low = mid + 1;
    else high = mid - 1;
}
return -1;
}

```

How Binary Search works:

- Array must be sorted. Compare key with mid element.
- If key == arr[mid] → found. If key > arr[mid] → search right half. Else search left half.
- Each iteration halves the search space → logarithmic behaviour.

Case	Condition	No. of Comparisons	Complexity	Notation Used
Best Case	Key found at arr[mid] on very first comparison	1	$O(1)$	$\Omega(1)$ and $O(1) \rightarrow \Theta(1)$
Average Case	Key found after approximately $\log n/2$ comparisons	$\approx \log n / 2$	$O(\log n)$	$\Theta(\log n)$
Worst Case	Key absent OR found at very last step; search space $\rightarrow 1$	$\log n + 1$	$O(\log n)$	$O(\log n)$

Recurrence Relation & Master Theorem:

$T(n) = T(n/2) + O(1) \rightarrow a=1, b=2, f(n)=O(1)=O(n^0)$

$\log_b(a) = \log_2(1) = 0 = \text{degree of } f(n) \rightarrow \text{Master Theorem Case 2} \rightarrow T(n) = O(\log n)$

Dry Run Trace on arr=[2, 5, 8, 12, 16, 23, 38], key=23:

Iteration	low	high	mid	arr[mid]	Comparison	Action
1	0	6	3	12	$23 > 12$	low = 4
2	4	6	5	23	$23 == 23$	Found at index 5 ✓

For key=99 (not present): low eventually > high → return -1. Worst case = 3 iterations = $\log_2 7 + 1 = 3$. ✓

◆ *Examiner's Note: Draw the trace table first — it typically carries 2 marks independently.*

c) Asymptotic Notation — Formal Proofs & Properties

[5MM]

Formal Definitions (must state before proofs):

- $f(n) = O(g(n))$ if $\exists$ constants $c > 0, n_0 > 0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$
- $f(n) = \Omega(g(n))$ if $\exists$ constants $c > 0, n_0 > 0$ such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$
- $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ AND $f(n) = \Omega(g(n))$ simultaneously

(i) $f(n) = 3n^2 + 5n + 7$ vs $g(n) = n^2$

Claim: $f(n) = \Theta(n^2)$

Proof of $O(n^2)$ — Upper bound:

$$3n^2 + 5n + 7 \leq 3n^2 + 5n^2 + 7n^2 \text{ for } n \geq 1$$

$$= 15n^2$$

So: $f(n) \leq 15 \cdot g(n)$ with $c=15, n_0=1 \rightarrow f(n) = O(n^2)$ ✓

Proof of $\Omega(n^2)$ — Lower bound:

$$3n^2 + 5n + 7 \geq 3n^2 \text{ for all } n \geq 1$$

So: $f(n) \geq 3 \cdot g(n)$ with $c=3, n_0=1 \rightarrow f(n) = \Omega(n^2)$ ✓

Since both O and Ω hold: $f(n) = \Theta(n^2)$ ✓

(ii) $f(n) = 2^n$ vs $g(n) = n^3$

Claim: $f(n) = \Omega(n^3)$ i.e., exponential dominates polynomial

Verify at $n=10$: $2^{10} = 1024$ and $10^3 = 1000 \rightarrow 2^n \geq n^3$ at $n=10$
For all $n \geq 10$: $2^n \geq 1 \cdot n^3 \rightarrow c=1, n_0=10 \rightarrow f(n) = \Omega(n^3)$ ✓
Also $2^n \neq O(n^3)$ because no constant c makes $2^n \leq c \cdot n^3$ for all large n .
Conclusion: 2^n grows FASTER than $n^3 \rightarrow f(n) = \Omega(n^3)$

(iii) $f(n) = \log n$ vs $g(n) = \sqrt{n}$

Claim: $f(n) = O(\sqrt{n})$ i.e., $\sqrt{n}$ grows faster than $\log n$

For $n=16$: $\log_2(16)=4$ and $\sqrt{16}=4 \rightarrow$ equal at $n=16$
For $n=256$: $\log_2(256)=8$ and $\sqrt{256}=16 \rightarrow \sqrt{n} > \log n$
So: $\log n \leq 1 \cdot \sqrt{n}$ for $n \geq 1 \rightarrow c=1, n_0=1 \rightarrow \log n = O(\sqrt{n})$ ✓

Two Properties of Big-O:

Property	Statement	Example
Transitivity	If $f=O(g)$ and $g=O(h)$ then $f=O(h)$	$n=O(n^2)$ and $n^2=O(n^3) \rightarrow n=O(n^3)$
Sum rule	$O(f+g) = O(\max(f,g))$	$O(n^2+n) = O(n^2)$

Two Properties of Big-Theta:

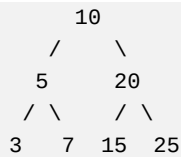
Property	Statement	Example
Symmetry	$f=\Theta(g) \iff g=\Theta(f)$	$n^2=\Theta(n^2) \leftrightarrow n^2=\Theta(n^2)$
Reflexivity	$f(n)=\Theta(f(n))$ always	$n^3=\Theta(n^3)$

♦ *Examiner's Note: Always state c and n_0 in proofs — that is the formal requirement for full marks.*

a) Binary Tree Traversals — Inorder, Preorder, Postorder

[5MM]

Given Binary Tree:



Definition of each traversal:

Traversal	Visit Order	Use Case
Inorder	Left → Root → Right	Produces sorted sequence for BST
Preorder	Root → Left → Right	Used to copy/serialize a tree
Postorder	Left → Right → Root	Used to delete a tree / evaluate expressions

Step-by-step Inorder (Left→Root→Right):

Start at root 10.

Go left → 5. Go left → 3. 3 has no left → Visit 3.

Back to 5 → Visit 5. Go right → 7 → Visit 7.

Back to 10 → Visit 10. Go right → 20.

Go left → 15 → Visit 15. Back to 20 → Visit 20. Go right → 25 → Visit 25.

Inorder Result: 3, 5, 7, 10, 15, 20, 25

Step-by-step Preorder (Root→Left→Right):

Visit 10 (root). Go left → Visit 5. Go left → Visit 3 (leaf).

Back to 5, go right → Visit 7 (leaf).

Back to 10, go right → Visit 20. Go left → Visit 15. Go right → Visit 25.

Preorder Result: 10, 5, 3, 7, 20, 15, 25

Step-by-step Postorder (Left→Right→Root):

Go to leftmost leaf: Visit 3. Back to 5, right → Visit 7. Now visit 5.

Go to 20's subtree: Visit 15, Visit 25, then Visit 20.

Finally visit root 10.

Postorder Result: 3, 7, 5, 15, 25, 20, 10

Traversal	Result Sequence
Inorder	3, 5, 7, 10, 15, 20, 25
Preorder	10, 5, 3, 7, 20, 15, 25
Postorder	3, 7, 5, 15, 25, 20, 10

Why Inorder gives sorted output for BST:

In a BST, every node satisfies: all keys in left subtree < node.key < all keys in right subtree. Inorder traversal visits left subtree first (smaller keys), then root, then right subtree (larger keys). This naturally produces keys in ascending order. For the given tree: **3 < 5 < 7 < 10 < 15 < 20 < 25 ✓**

◆ *Examiner's Note: Time complexity of all traversals: $O(n)$. Space: $O(h)$ where h =height (recursion stack).*

b) AVL Tree Construction — Insertions & Rotations

[5MM]

AVL Tree Definition:

An AVL tree is a self-balancing BST where for every node, $|\text{height}(\text{left subtree}) - \text{height}(\text{right subtree})| \leq 1$. This is the Balance Factor (BF). Valid BF values: $\{-1, 0, +1\}$.

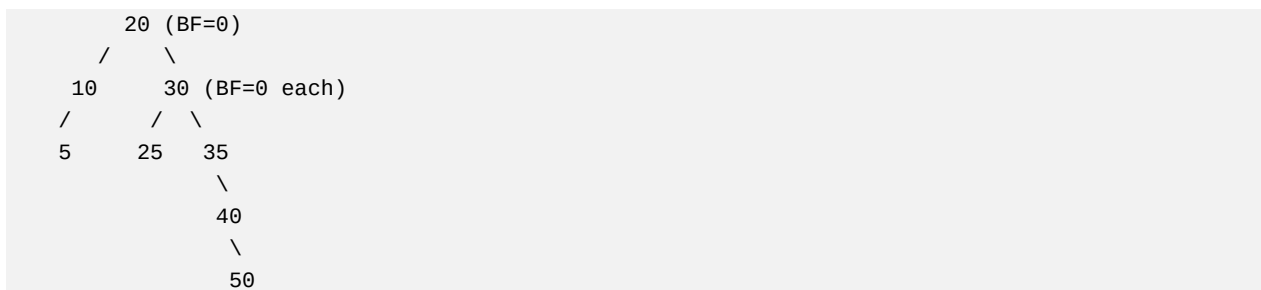
Four Rotation Types:

Rotation	Imbalance at	Trigger	Fix
LL (Right Rotate)	Node z	New node in LEFT subtree of LEFT child	Right-rotate at z
RR (Left Rotate)	Node z	New node in RIGHT subtree of RIGHT child	Left-rotate at z
LR (Left then Right)	Node z	New node in RIGHT subtree of LEFT child	Left-rotate at y, then Right-rotate at z
RL (Right then Left)	Node z	New node in LEFT subtree of RIGHT child	Right-rotate at y, then Left-rotate at z

Insert keys: 30, 20, 40, 10, 25, 35, 50, 5

#	Insert	Tree State (BF check)	Rotation Needed	Result
1	30	Root=30. BF=0. Balanced.	None	[30]
2	20	20<30 → left. BF(30)=+1. OK.	None	30(L:20)
3	40	40>30 → right. BF(30)=0. OK.	None	30(L:20, R:40)
4	10	10<30<20 → left of 20. BF(20)=+1, BF(30)=+2. IMBALANCE!	LL Rotation at 30	20 becomes root; 30 goes right; 10 stays left
5	25	25>20, <30. Right child of 20. BF(20)=0, BF(30)=+1. OK.	None	20(L:10, R:30); 30(L:25, R:40)
6	35	35>30, <40. Left of 40. BF(40)=+1. OK.	None	Subtree balanced
7	50	50>40. BF(40)=-2. IMBALANCE!	RR Rotation at 40	35 becomes new local root; 40 goes right; 50 stays
8	5	5<10<20. BF(10)=+1. OK overall.	None	Final tree balanced

Final AVL Tree structure:



♦ *Examiner's Note: Draw the tree after EACH rotation — examiners award 1 mark per correctly drawn intermediate state.*

c) Threaded Binary Tree — Concept, Types & Inorder Traversal Algorithm

[5MM]

What is a Threaded Binary Tree?

In a regular binary tree with n nodes, there are exactly $n+1$ **NULL pointers** (wasted space). A Threaded Binary Tree utilises these NULL pointers by replacing them with **threads** — pointers to the inorder predecessor or successor of a node. This eliminates the need for a stack or recursion during traversal.

Node Structure for Threaded Binary Tree:

```

struct Node {
    int data;
    Node* left;
    Node* right;
    bool leftThread; // true if left points to inorder predecessor
    bool rightThread; // true if right points to inorder successor
};

```

Feature	Single-Threaded TBT	Double-Threaded TBT
NULL replaced	Only right NULL → inorder successor	Both left NULL → predecessor AND right NULL → successor
Boolean flags	One flag per node (rightThread)	Two flags per node (leftThread + rightThread)
Space saved	Partial	Maximum — all NULL pointers used
Traversals	Inorder (forward) only	Inorder and Reverse-Inorder
Complexity	Simpler to implement	More complex but more flexible

Algorithm — Inorder Traversal of Right-Threaded Binary Tree:

```

Algorithm: InorderRightThreaded(root)

Step 1: curr = leftMost(root)
// leftMost: keep going left until no real left child

Step 2: WHILE curr != NULL:
PRINT curr.data

IF curr.rightThread == TRUE:
// Right pointer is a thread → follow it to inorder successor
curr = curr.right
ELSE:
// Right pointer is a real link → go to leftmost of right subtree
curr = leftMost(curr.right)

Function leftMost(node):
WHILE node != NULL AND node.leftThread == FALSE:
node = node.left
RETURN node

```

Example trace on Right-Threaded Tree [3,5,7,10,15,20,25]:

curr	rightThread?	Action	Output so far
3 (leftmost)	TRUE	Follow right thread → 5	3
5	FALSE	Go to leftMost(7) = 7	3, 5
7	TRUE	Follow right thread → 10	3, 5, 7
10	FALSE	Go to leftMost(20's left=15) = 15	3, 5, 7, 10
15	TRUE	Follow right thread → 20	3, 5, 7, 10, 15
20	FALSE	Go to leftMost(25) = 25	3, 5, 7, 10, 15, 20
25	TRUE	right = NULL → STOP	3, 5, 7, 10, 15, 20, 25 ✓

Advantages over regular traversal:

- No stack or recursion needed → $O(1)$ extra space instead of $O(h)$
- Faster traversal — no function call overhead
- Utilises wasted NULL pointers → memory efficient

♦ *Examiner's Note: Draw the threaded tree with dashed arrows for threads — this diagram earns 2 marks on its own.*

[5MM]

Building BST from: 30, 20, 40, 10, 25

```

      30
     /  \
    20   40
   /  \
  10   25

```

```
struct Node {  
    int key; Node *left, *right;  
    Node(int k): key(k), left(NULL), right(NULL){}  
};  
  
// ■■ INSERT  
Node* insert(Node* root, int k) {  
    if (!root) return new Node(k);  
    if (k < root->key) root->left = insert(root->left, k);  
    else if (k > root->key) root->right = insert(root->right, k);  
    // if k == root->key: duplicate, ignore  
    return root;  
}  
  
// ■■ SEARCH  
Node* search(Node* root, int k) {  
    if (!root || root->key == k) return root;  
    if (k < root->key) return search(root->left, k);  
    return search(root->right, k);  
}  
  
// ■■ HELPER: find minimum in subtree  
Node* minNode(Node* n) {  
    while (n->left) n = n->left;  
    return n;  
}  
  
// ■■ DELETE  
Node* deleteNode(Node* root, int k) {  
    if (!root) return root;  
    if (k < root->key) root->left = deleteNode(root->left, k);  
    else if (k > root->key) root->right = deleteNode(root->right, k);  
    else {  
        // Case 1: Leaf node (no children)  
        if (!root->left && !root->right) { delete root; return NULL; }  
        // Case 2: One child  
        if (!root->left) { Node* t=root->right; delete root; return t; }  
        if (!root->right) { Node* t=root->left; delete root; return t; }  
        // Case 3: Two children → replace with inorder successor  
        Node* succ = minNode(root->right); // leftmost of right subtree  
        root->key = succ->key; // copy successor's key
```

```

    root->right = deleteNode(root->right, succ->key); // delete successor
}
return root;
}

```

Trace — Delete key 20 (two children case):

Step	Action
1	Search for 20 in BST → found at left child of root 30
2	Node 20 has two children: left=10, right=25 → Case 3 applies
3	Find inorder successor = leftmost node in right subtree of 20 = 25 (no left child)
4	Copy 25 into node 20's position → node now holds key 25
5	Delete node 25 from right subtree (leaf deletion — Case 1)
6	BST after deletion: 30(L:25, R:40); 25(L:10)

BST after deleting 20:

```

    30
   /  \
  25   40
  /
 10

```

Operation	Average Case	Worst Case (Skewed)
Insert	$O(\log n)$	$O(n)$
Search	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

♦ *Examiner's Note: Always handle all 3 deletion cases — leaf, one child, two children. Examiners check all three.*

b) Min-Heap — Array Implementation, Insert (Heapify-Up) & Extract-Min

[5MM]

Min-Heap Definition:

A complete binary tree where every parent node $\leq$ both its children. The minimum element is always at the root. Stored compactly as an array.

Array Index Formulas:

Relationship	Formula
Left child of node i	$\text{index} = 2i + 1$
Right child of node i	$\text{index} = 2i + 2$
Parent of node i	$\text{index} = (i-1) / 2$

Step-by-step insertion of [15, 10, 8, 12, 5, 3]:

Step	Insert	Array State	Heapify-Up Action	Final Array
1	15	[15]	Root — no parent to compare	[15]
2	10	[15,10]	10 at idx 1; parent=15 at idx 0; $10 < 15$ → SWAP	[10, 15]
3	8	[10,15,8]	8 at idx 2; parent=10 at idx 0; $8 < 10$ → SWAP	[8, 15, 10]
4	12	[8,15,10,12]	12 at idx 3; parent=15 at idx 1; $12 < 15$ → SWAP; parent=8; $12 > 8$ → STOP	[8, 12, 10, 15]

5	5	[8,12,10,15,5]	5 at idx 4; parent=12 at idx 1; 5<12→SWAP; parent=8 at idx 0; 5<8→SWAP	[5, 8, 10, 15, 12]
6	3	[5,8,10,15,12,3]	3 at idx 5; parent=10 at idx 2; 3<10→SWAP; parent=5 at idx 0; 3<5→SWAP	[3, 8, 5, 15, 12, 10]

Final Min-Heap after all insertions — Array: [3, 8, 5, 15, 12, 10]

```

      3 ← root (minimum)
     /  \
    8    5
   /  \  /
  15 12 10

```

Extract-Min Operation (removes root = minimum):

Step	Action	Array State
1	Save root value (min = 3)	[3, 8, 5, 15, 12, 10]
2	Move last element (10) to root position	[10, 8, 5, 15, 12]
3	Heapify-Down: compare 10 with children 8(idx1) and 5(idx2); smallest=5; 10>5 → SWAP	[5, 8, 10, 15, 12]
4	10 is now at idx 2; children would be idx 5,6 (don't exist); STOP	[5, 8, 10, 15, 12]
5	Return saved value 3	Min extracted = 3 ✓

Operation	Time Complexity	Space Complexity
Insert (heapify-up)	O(log n)	O(1)
Extract-Min (heapify-down)	O(log n)	O(1)
Build Heap from n elements	O(n)	O(1)
Peek min (root)	O(1)	O(1)

♦ *Examiner's Note: Heap as Priority Queue: elements served by priority not arrival order. Min-heap → smallest priority served first.*

c) B-Tree vs B+ Tree — Comparison & B+ Tree Insertion Trace

[5MM]

B-Tree and B+ Tree — Detailed Comparison:

Feature	B-Tree	B+ Tree
Data storage	Data stored in ALL nodes (both internal and leaf)	Data stored ONLY in leaf nodes; internal nodes store keys for routing only
Internal nodes	Contain keys + associated data	Contain only keys (act as index/guide)
Leaf nodes	Standalone — not linked to each other	Linked as a doubly linked list (enables fast range queries)
Search	May terminate at any level when key found	Always goes to leaf level
Range queries	Slow — requires tree traversal	Very fast — just scan leaf linked list
Height	Slightly smaller (data in all nodes)	Slightly taller
Space	Less efficient per level	More efficient — all data at one level
Real usage	General B-Tree indexes	MySQL InnoDB, PostgreSQL, NTFS, ext4

B+ Tree of Order 3 — Properties:

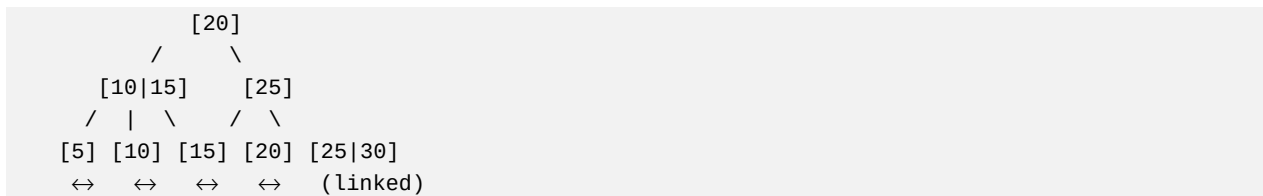
- Maximum keys per node = order-1 = 2

- Maximum children per internal node = 3
- Split occurs when a node accumulates 3 keys (overflow)
- In B+ Tree: pushed-up key is COPIED to parent, original stays in leaf

Insert keys: 5, 10, 15, 20, 25, 30 — step by step:

After inserting	Tree structure	Split?
5	Leaf: [5]	No
10	Leaf: [5 10]	No
15 ← OVERFLOW	Leaf [5 10 15] has 3 keys → split Push middle key (10) up to parent Left leaf: [5] Right leaf: [10 15]	YES — Push 10 up Root: [10]
20	20>10 → right leaf [10 15] → [10 15 20] → overflow Push 15 up. Left:[10] Right:[15 20]	YES — Push 15 up Root: [10 15]
25	25>15 → rightmost leaf [15 20] → [15 20 25] → overflow Push 20 up. Right leaf splits: [15] and [20 25]	YES — Push 20 up Root: [10 15 20]
30 ← Root splits	30 → [20 25 30] overflow → push 25 up Root [10 15 20 25] overflow → split root New root: [20] Left sub-root: [10 15] Right sub-root: [25] Leaves: [5]↔[10]↔[15]↔[20]↔[25 30]	YES — Root split Tree grows taller

Final B+ Tree after inserting 5,10,15,20,25,30:



♦ *Examiner's Note: Key rule: In B+ Tree internal nodes contain COPIES of keys. Leaves form a sorted linked list — always draw the ↔ links.*

a) Dijkstra's Single Source Shortest Path Algorithm

[5MM]

Graph: Vertices {A,B,C,D,E,F} | Edges: A-B(4), A-C(2), B-C(1), B-D(5), C-D(8), C-E(10), D-F(2), E-F(3)

Algorithm Steps:

- Initialize $\text{dist}[\text{source}] = 0$ and $\text{dist}[\text{all others}] = \infty$
- Use a min-priority queue; extract vertex u with minimum distance
- For each neighbor v of u : if $\text{dist}[u] + \text{weight}(u,v) < \text{dist}[v] \rightarrow$ update $\text{dist}[v]$ (relaxation)
- Mark u as visited; repeat until all vertices processed

Execution starting from A:

Step	Vertex Extracted	dist[A]	dist[B]	dist[C]	dist[D]	dist[E]	dist[F]	Edges Relaxed
Init	—	0	∞	∞	∞	∞	∞	—
1	A (d=0)	0	4	2	∞	∞	∞	A→B: $\min(\infty, 0+4)=4$ A→C: $\min(\infty, 0+2)=2$
2	C (d=2)	0	3	2	10	12	∞	C→B: $\min(4, 2+1)=3$ ✓ C→D: $\min(\infty, 2+8)=10$ C→E: $\min(\infty, 2+10)=12$
3	B (d=3)	0	3	2	8	12	∞	B→D: $\min(10, 3+5)=8$ ✓ (B→C already visited)
4	D (d=8)	0	3	2	8	12	10	D→F: $\min(\infty, 8+2)=10$ ✓
5	F (d=10)	0	3	2	8	13	10	F→E: $\min(12, 10+3)=13$ ✓
6	E (d=13)	0	3	2	8	13	10	No new relaxations

Final Shortest Distances and Paths from A:

Destination	Shortest Distance	Shortest Path	Verification
B	3	A → C → B	A-C(2) + C-B(1) = 3 ✓ (shorter than direct A-B=4)
C	2	A → C	Direct A-C(2) ✓
D	8	A → C → B → D	2+1+5 = 8 ✓
E	13	A → C → B → D → F → E	2+1+5+2+3 = 13 ✓
F	10	A → C → B → D → F	2+1+5+2 = 10 ✓

Key Notes:

- Dijkstra's finds SHORTEST PATH from one source to all others — NOT MST
- Works only for non-negative edge weights
- Time Complexity: $O(V^2)$ with array; $O((V+E) \log V)$ with min-heap
- For negative weights use Bellman-Ford algorithm instead

♦ *Examiner's Note: The relaxation step $\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w(u,v))$ must be written explicitly — it earns marks.*

b) Kruskal's Algorithm — MST using Union-Find + Comparison with Prim's

[5MM]

Same Graph: Edges: A-B(4), A-C(2), B-C(1), B-D(5), C-D(8), C-E(10), D-F(2), E-F(3)

Step 1 — Sort all edges by weight (ascending):

Rank	Edge	Weight
------	------	--------

1	B-C	1
2	A-C	2
3	D-F	2
4	E-F	3
5	A-B	4
6	B-D	5
7	C-D	8
8	C-E	10

Step 2 — Union-Find: process edges in order, skip if cycle formed:

Initial sets (each vertex alone): {A}, {B}, {C}, {D}, {E}, {F}

Edge	Weight	find(u)	find(v)	Same set?	Action	Sets after	MST Cost
B-C	1	B	C	No → union	Add B-C ✓	{B,C},{A},{D},{E},{F}	1
A-C	2	A	C	No → union	Add A-C ✓	{A,B,C},{D},{E},{F}	3
D-F	2	D	F	No → union	Add D-F ✓	{A,B,C},{D,F},{E}	5
E-F	3	E	F	No → union	Add E-F ✓	{A,B,C},{D,E,F}	8
A-B	4	A	B	YES	Skip (cycle!)	unchanged	8
B-D	5	B	D	No → union	Add B-D ✓	{A,B,C,D,E,F}	13
—	—	—	—	V-1=5 edges	STOP	All connected	13

MST Edges: B-C(1), A-C(2), D-F(2), E-F(3), B-D(5)

Total MST Cost = 1+2+2+3+5 = 13

MST Diagram:

```

A(2)---C(1)---B(5)---D(2)---F
      |
      (3)
      E

```

Feature	Kruskal's Algorithm	Prim's Algorithm
Approach	Edge-based: sort ALL edges globally	Vertex-based: grow MST from one source vertex
Data Structure	Union-Find (Disjoint Set Union)	Min Priority Queue / visited boolean array
Starting	No fixed start — processes all edges	Must start from a specific vertex
Best suited	Sparse graphs ($E \ll V^2$)	Dense graphs ($E \approx V^2$)
Complexity	$O(E \log E)$ — dominated by sorting	$O(V^2)$ with array; $O(E \log V)$ with heap
Cycle detection	Union-Find checks if same component	Implicit — only MST-to-non-MST edges added
MST cost	Same optimal result	Same optimal result

♦ *Examiner's Note: Both Kruskal's and Prim's always give the same MST COST. Edge selection order may differ.*

c) Prim's Algorithm — Step-by-Step MST from Vertex A

[5MM]

Same Graph: Edges: A-B(4), A-C(2), B-C(1), B-D(5), C-D(8), C-E(10), D-F(2), E-F(3)

Prim's Algorithm Logic:

- Start: Pick any vertex (say A), add to MST set
- At each step: find the minimum weight edge with exactly ONE endpoint inside MST set
- Add that edge and the new vertex to MST set
- Repeat until all V vertices are in MST set (V-1 edges added)

Execution starting from A:

Step	MST Set	All Candidate Edges (from inside→outside)	Minimum Edge Chosen	Cost Added	Total MST Cost
Start	{A}	A-B(4), A-C(2)	A-C(2)	2	2
1	{A,C}	A-B(4), C-B(1), C-D(8), C-E(10)	C-B(1)	1	3
2	{A,C,B}	A-B(skip-already in), B-D(5), C-D(8), C-E(10)	B-D(5)	5	8
3	{A,C,B,D}	D-F(2), C-D(skip), C-E(10)	D-F(2)	2	10
4	{A,C,B,D,F}	F-E(3), C-E(10)	F-E(3)	3	13
5	{A,C,B,D,F,E}	All V=6 vertices included — STOP	—	—	13

MST Edges selected by Prim's: A-C(2), C-B(1), B-D(5), D-F(2), F-E(3)

Total MST Cost = 2+1+5+2+3 = 13 (same as Kruskal's ✓)

Prim's MST Diagram:

```

A(2)---C(1)---B(5)---D(2)---F
          |
          (3)
          E
  
```

Why is Prim's better for dense graphs?

In a dense graph, $E \approx V^2$. Kruskal's must sort all E edges $\rightarrow O(E \log E) = O(V^2 \log V)$. Prim's with a min-heap processes only V extractions and E edge relaxations $\rightarrow O(E \log V) = O(V^2 \log V)$ but with better constant factors. With a Fibonacci heap, Prim's achieves $O(E + V \log V)$.

What data structure improves Prim's efficiency?

Data Structure	Complexity of Prim's	When to use
Simple array (linear scan)	$O(V^2)$	Small dense graphs
Binary Min-Heap	$O(E \log V)$	General use — most efficient in practice
Fibonacci Heap	$O(E + V \log V)$	Theoretical optimal — complex to implement

◆ *Examiner's Note: Remember: Prim's is for MST (minimum cost spanning tree), NOT shortest path. Dijkstra's is for shortest path. Both use a min-heap but with different goals.*

a) Hash Table — Linear, Quadratic & Chaining Comparison

[5MM]

Hash Table Size = 11 | Hash Function: $h(k) = k \bmod 11$

Keys to insert: 22, 33, 44, 55, 66, 77, 88

Note: $22 \bmod 11 = 0$, $33 \bmod 11 = 0$, $44 \bmod 11 = 0$... all keys hash to index 0!(i) Linear Probing — probe: $(h(k) + i) \bmod 11$

Key	$h(k)$	Probe Sequence	Final Index	Probes Taken
22	0	0	0	1 (no collision)
33	0	0(busy)→1	1	2
44	0	0,1(busy)→2	2	3
55	0	0,1,2(busy)→3	3	4
66	0	0,1,2,3(busy)→4	4	5
77	0	0,1,2,3,4(busy)→5	5	6
88	0	0,1,2,3,4,5(busy)→6	6	7

Total collisions = 6 | Problem: **Primary Clustering** — long occupied runs form(ii) Quadratic Probing — probe: $(h(k) + i^2) \bmod 11$

Key	$h(k)$	Probe: $i=0,1,2,3 \dots \rightarrow (h+i^2) \bmod 11$	Final Index
22	0	0	0
33	0	0(busy), $(0+1)=1$	1
44	0	0,1(busy), $(0+4)=4$	4
55	0	0,1,4(busy), $(0+9)=9$	9
66	0	0,1,4,9(busy), $(0+16) \bmod 11 = 5$	5
77	0	0,1,4,9,5(busy), $(0+25) \bmod 11 = 3$	3
88	0	0,1,4,9,5,3(busy), $(0+36) \bmod 11 = 3$ (busy)→ $(0+49) \bmod 11 = 5$ (busy)→continue	6

Total collisions = 6 | Problem: **Secondary Clustering** — same $h(k) \rightarrow$ same probe path

(iii) Chaining — each index holds a linked list

Index	Chain Content
0	22 → 33 → 44 → 55 → 66 → 77 → 88 → NULL
1 to 10	Empty (NULL)

Collisions = 6 | Advantage: no table overflow; unlimited keys per index; simple deletion

Criteria	Linear Probing	Quadratic Probing	Chaining
Collision type	Primary clustering	Secondary clustering	No clustering
Extra memory	None	None	Linked list pointers
Deletion	Complex (need tombstone)	Complex	Simple
Cache performance	Best (contiguous)	Good	Poor (pointers)
Load factor limit	< 0.7 recommended	< 0.5 recommended	Any load factor
Best when	Low load factor	Medium load	High load/many collisions

♦ *Examiner's Note: These keys all hash to 0 — worst case. Always mention this observation. Define load factor $\alpha = n/m$; performance degrades as $\alpha \rightarrow 1$ for open addressing.*

b) Double Hashing — Complete Probe Sequence & Final Table

[5MM]

$h_1(k) = k \bmod 7$ | $h_2(k) = 5 - (k \bmod 5)$ | Table size = 7 (prime)

Probe formula: $\text{index} = (h_1(k) + i \times h_2(k)) \bmod 7$ where $i = 0, 1, 2, \dots$

Why table size must be prime: Ensures every slot is reachable in probe sequence.

Why $h_2(k)$ must never be 0: If $\text{step}=0$, probe never moves $\rightarrow$ infinite loop.

Insert keys: 18, 41, 22, 44, 59, 32, 31, 73

Key	$h_1(k) = k \bmod 7$	$h_2(k) = 5 - (k \bmod 5)$	$i=0$	$i=1 (h_1+h_2)$	$i=2 (h_1+2h_2)$	Final Index	Table after
18	4 (18mod7)	2 (5-18mod5 =5-3)	4 (free)	—	—	4	[_,_,_,18,_,_]
41	6 (41mod7)	4 (5-41mod5 =5-1)	6 (free)	—	—	6	[_,_,_,18,_,41]
22	1 (22mod7)	3 (5-22mod5 =5-2)	1 (free)	—	—	1	[_,22,_,_,18,_,41]
44	2 (44mod7)	1 (5-44mod5 =5-4)	2 (free)	—	—	2	[_,22,44,_,_,18,_,41]
59	3 (59mod7)	1 (5-59mod5 =5-4)	3 (free)	—	—	3	[_,22,44,59,_,18,_,41]
32	4 (32mod7)	3 (5-32mod5 =5-2)	4 (busy $\rightarrow$ 18)	(4+3)=0	—	0	[32,22,44,59,18,_,41]
31	3 (31mod7)	4 (5-31mod5 =5-1)	3 (busy $\rightarrow$ 59)	(3+4)=0(busy)	(3+8)=4(busy)	—(table full for many)	5
73	3 (73mod7)	2 (5-73mod5 =5-3)	3 (busy)	(3+2)=5(busy $\rightarrow$ 31)	(3+4)=0(busy)	—	Overflow

Final Hash Table (size 7):

Index	0	1	2	3	4	5	6
Key	32	22	44	59	18	31	41

Why Double Hashing is the best open addressing method:

Property	Linear Probing	Quadratic Probing	Double Hashing
Probe step size	Always 1 (fixed)	i^2 (varies but same for same $h(k)$)	$i \times h_2(k)$ (varies per key)
Primary clustering	YES — severe	No	No
Secondary clustering	No	YES	No
Distribution quality	Poor	Medium	Best
Cache performance	Best	Good	Slightly worse
Complexity	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg

♦ *Examiner's Note: Key requirements for double hashing: (1) table size must be prime, (2) $h_2(k)$ must never return 0, (3) h_1 and h_2 must be independent functions.*

c) Java Collections (HashMap+TreeMap) + Skip List with Search Trace

[5MM]

Part (i) — Java Program: HashMap + TreeMap for Student Marks

```
import java.util.*;

public class StudentMarks {
    public static void main(String[] args) {

        // Step 1: Store student data using HashMap (rollNo → marks)
        HashMap<Integer, Integer> hmap = new HashMap<>();
        hmap.put(101, 85);
        hmap.put(102, 90);
        hmap.put(103, 78);
        hmap.put(104, 92);
        hmap.put(105, 88);

        // Step 2: Update marks for roll no 103
        hmap.put(103, 95); // put() overwrites if key already exists
        System.out.println("Updated marks for 103: " + hmap.get(103));

        // Step 3: Display all entries SORTED by roll number using TreeMap
        // TreeMap is backed by Red-Black Tree → maintains sorted key order
        TreeMap<Integer, Integer> tmap = new TreeMap<>(hmap);
        System.out.println("\nRoll No\tMarks");
        System.out.println("-----");
        for (Map.Entry<Integer, Integer> e : tmap.entrySet()) {
            System.out.println(e.getKey() + "\t" + e.getValue());
        }

        // Additional operations
        System.out.println("First roll: " + tmap.firstKey()); // 101
        System.out.println("Last roll: " + tmap.lastKey()); // 105
    }
}
```

Output:

```
Updated marks for 103: 95
Roll No  Marks
-----
101      85
102      90
103      95 ← updated
104      92
105      88
First roll: 101
Last roll: 105
```

Feature	HashMap	TreeMap	HashSet
Internal structure	Hash table	Red-Black Tree	Hash table
Key ordering	No guaranteed order	Sorted (natural or comparator)	No order
get/put complexity	O(1) average	O(log n)	O(1) average
Null keys	1 null key allowed	Not allowed	1 null allowed
Use case	Fast lookup	Sorted traversal needed	No duplicates

Part (ii) — Skip List for keys [3, 7, 12, 19, 26] with 3 levels

What is a Skip List:

A Skip List is a probabilistic data structure built as multiple layers of linked lists. The bottom layer (Level 0) contains all elements in sorted order. Each higher level contains a subset of elements — each node is promoted to the next

level with probability $p = 0.5$. This allows $O(\log n)$ expected search time without tree balancing.

Skip List Diagram (3 levels):



Search for key 19 — Step-by-step trace:

Step	Level	Current Node	Next Node (value)	Comparison	Action
1	2 (top)	HEAD	12	$12 < 19$	Move RIGHT to 12
2	2	12	NIL	NIL encountered	Drop DOWN to Level 1
3	1	12	19	$19 == 19$	Move RIGHT to 19
4	1	19	NIL	At search key, confirm	Drop DOWN to Level 0
5	0	19	26	$19 == 19 \checkmark$	KEY FOUND at Level 0

Search Result: Key 19 found in 5 steps (instead of 5 linear steps — same here, but for larger data, skip list is much faster)

Operation	Skip List	Sorted Linked List	Balanced BST
Search	$O(\log n)$ expected	$O(n)$	$O(\log n)$
Insert	$O(\log n)$ expected	$O(n)$	$O(\log n)$
Delete	$O(\log n)$ expected	$O(n)$	$O(\log n)$
Space	$O(n \log n)$ expected	$O(n)$	$O(n)$
Implementation	Simple	Very simple	Complex (rotations)

♦ *Examiner's Note: Skip List is randomized — no worst-case $O(\log n)$ guarantee (unlike AVL/Red-Black). Advantage: no rotations needed; easy to implement concurrent version.*

★ ★ ★ END OF ANSWER KEY — ALL THE BEST ★ ★ ★

Total: 5 Questions × 2 Sub-questions × 5 Marks = 50 Marks